Projekt: Objektorientierte und funktionale Programmierung in Python - Konzeptionsphase

Lukas Latschbacher

17. April 2026

Inhaltsverzeichnis

1 Festlegen der Ziele **2**
1.1 1. Ziel . . . 2
1.2 2. Ziel . . . 2

2 Dashboard Prototyp **2**
2.1 Überschrift . . . 2
2.2 ECTS Counter . . . 2
2.3 ECTS pro Monat . . . 2
2.4 Zeit zu ECTS . . . 2
2.5 Hinzufügen Button . . . 3
2.6 Hinzufügen Seite . . . 3

3 UML-Klassendiagramm **3**

4 Machbarkeitsüberprüfung und Erprobung von Umsetzungsmöglichkeiten mit Python **4**
4.1 Entwicklungsumgebung und Bibliotheken . . . 4
4.1.1 Entwicklungsumgebung . . . 4
4.1.2 Bibliotheken . . . 4
4.2 Datenspeicherung . . . 4
4.3 Quellcodeverwaltung . . . 5
4.4 Interaktion mit dem User . . . 5
4.5 Testprogramme . . . 5

1 Festlegen der Ziele

1.1 1. Ziel

Pro Monat sollen mindestens 5 ECTS erlangt werden.

1.2 2. Ziel

Das Studium soll in mindestens 3 Jahre abgeschlossen sein.

2 Dashboard Prototyp

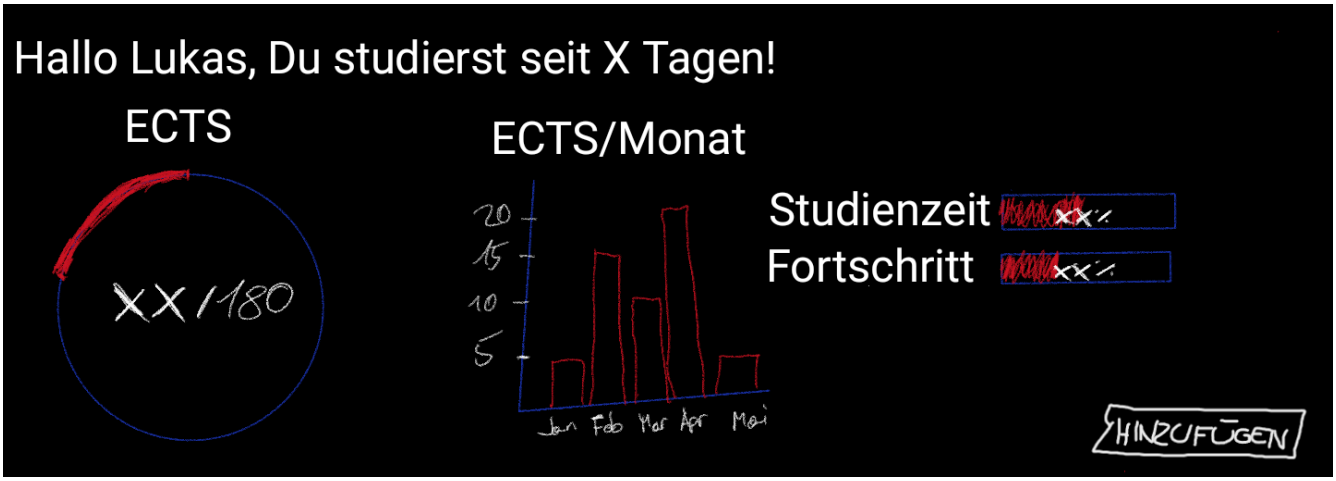

Abbildung 1: Skizze des Dashboards

2.1 Überschrift

In der Überschrift soll der User mit seinem Vornamen begrüßt und die Studienzeit in Tagen angezeigt werden, damit der/die User/User*in sieht wie lange das Studium gesamt schon läuft. Der Name soll das Dashboard etwas persönlicher gestalten.

2.2 ECTS Counter

Durch den ECTS Counter soll eine übersicht des Fortschritts in einer ECTS Zahl dargestellt werden.

2.3 ECTS pro Monat

Mit der Grafik für die ECTS pro Monat soll das Ziel mindestens 5 ECTS pro Monat zu erzielen überwacht werden. Mit dieser Grafik soll es auf einen Blick gelingen wie viele ECTS im dazugehörigen Monat erreicht wurden.

2.4 Zeit zu ECTS

Das Ziel das Studium in mindestens 3 Jahren abzuschließen erfordert eine Anzeige für die Zeit und eine für den Fortschritt. Diese beiden müssen gegenübergestellt werden, damit das Ziel überwacht werden kann. Beide Anzeigen zeigen die jeweiligen Werte in Prozent an, damit beides verglichen werden kann.

2.5 Hinzufügen Button

Durch den Button rechts unten soll es möglich sein erbrachte Prüfungsleistungen hinzuzufügen. Dabei soll in weiterer Folge ein Fenster aufgehen in dem der User die Daten für die abgeleistete Prüfungsleistung eintragen kann.

2.6 Hinzufügen Seite

In der folgenden Skizze ist die Seite zum Hinzufügen von Daten zu sehen. Bei dieser können Daten einer Prüfungsleistung eingegeben werden. Auszuwählen über Buttons ist die Note (1 - 5), in ein Textfeld einzugeben die ECTS, die diese Prüfungsleistung erbringt, die Nummer des Semesters einzugeben und in einem Kalender das Datum dieser Prüfungsleistung zu wählen. Das Datum soll standardmäßig auf das Datum des heutigen Tages gesetzt werden.

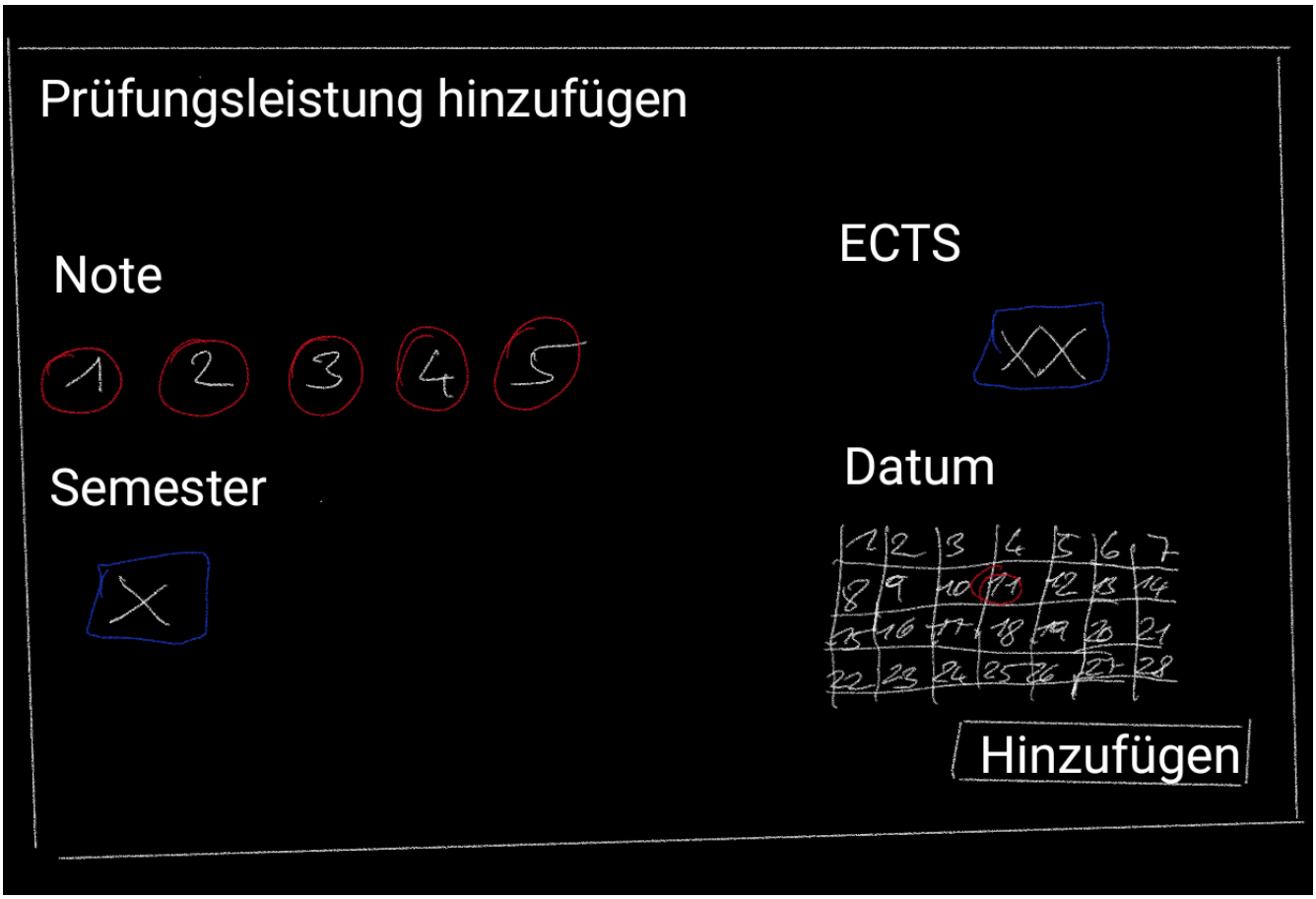

Abbildung 2: Skizze der Hinzufügen Seite

3 UML-Klassendiagramm

In diesem UML Diagramm sind 4 Klassen zu sehen. Erstens EIN Studium, welches aus MINDESTENS EINEM Semester besteht. Da das Semester nicht ohne einem Studium bestehen kann ist eine Komposition notwendig. Weiters gibt es noch die Klasse Modul, wobei EIN Semester 0..* Module hat. Diese sind mit einer Aggregation verbunden, da ein Modul auch statt in Semester 1 in Semester 3 angeboten werden kann. Die vierte Klasse ist die Prüfungsleistung. EIN Modul hat EINE Prüfungsleistung und diese sind mit einer Komposition verbunden, da die Prüfungsleistung nicht ohne das Modul bestehen kann.

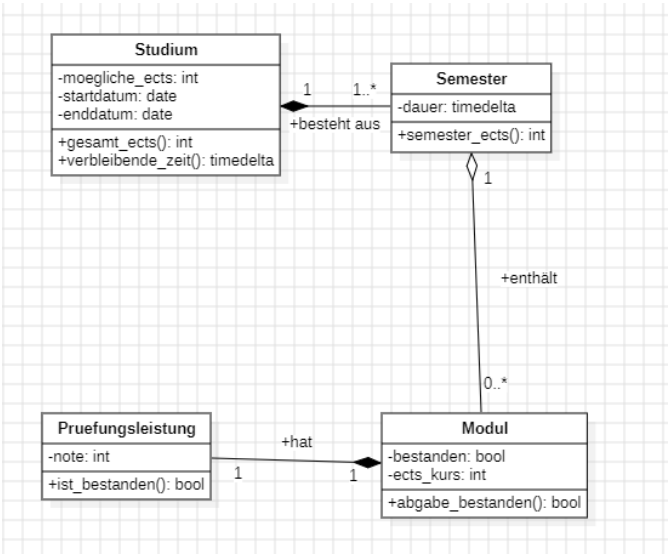

Abbildung 3: UML Diagramm in StarUML

4 Machbarkeitsüberprüfung und Erprobung von Umsetzungsmöglichkeiten mit Python

4.1 Entwicklungsumgebung und Bibliotheken

Welche Python Entwicklungsumgebung und welche Python Bibliotheken sollen warum zum Einsatz kommen?

4.1.1 Entwicklungsumgebung

Als Entwicklungsumgebung bietet sich PyCharm an, da dieses ein guter Allrounder im Bereich Python ist, integriertes Debugging anbietet, Unterstützung für Git besteht, Code Vervollständigung beinhaltet und kostenlos zur Verfügung steht.

4.1.2 Bibliotheken

Bibliothek für das GUI Als Bibliothek für die grafische Benutzeroberfläche bieten sich zwei verschiedene Bibliotheken an. Zum eine würde PyQt6 in Frage kommen, zum anderen Tkinter. PyQt6 hat zwar viele Funktionen, es werden die meisten davon jedoch nicht benötigt. Außerdem ist PyQt6 so umfangreich, dass es eine deutlich größere Lernkurve als Tkinter hat. Weiters ist der Dateiumfang deutlich größer, Tkinter ist deutlich leichtgewichtiger und reicht für kleine Anwendungen wie dieses Dashboard völlig aus. Tkinter erfordert auch keine Installationen im Gegensatz zu PyQt6. Ein kleiner Nachteil an Tkinter ist, dass die Entwicklerbasis nicht so groß ist wie bei PyQt6. Zusammengefasst ist Tkinter die bessere Lösung für dieses Projekt.

Bibliothek für die Graphen Da Tkinter keine Säulen und Ringdiagramme darstellen kann braucht es eine weitere Bibliothek. Dazu bietet sich Matplotlib an. Es ist zwar nicht die schlankeste Bibliothek, aber sie bietet Integration für Tkinter und alle gefragten Diagrammtypen.

4.2 Datenspeicherung

Frage: Soll die Datenspeicherung in Dateien oder in einer Datenbank erfolgen?
Da die Abgabe erfordert, dass das Programm auf einem PC laufen muss, ohne etwa Docker Container oder VMs zusätzlich aufzusetzen ist die Entscheidung vergleichsweise einfach. Die Daten sollen in einer Datei gespeichert werden, da dafür keine zusätzlichen Ressourcen erforderlich sind. Als Datentyp bietet sich eine JSON Datei an, da

sie die Verschachtelung gut Darstellen kann wie zum Beispiel Module in einem Semester. Die Datei bleibt dabei relativ gut lesbar für Menschen und JSON wird außerdem nativ von Python unterstützt.

4.3 Quellcodeverwaltung

Wie soll der Quellcode verwaltet werden?
Da Git eine direkte Anbindung an PyCharm anbietet ist es naheliegend dieses Versionierungssystem zu verwenden. Außerdem ist es das am weitesten verbreitetste Versionierungssystem, was für viel Community Support spricht, da bei den meisten Problem Lösungen in Foren bereitstehen.

4.4 Interaktion mit dem User

Soll die Interaktion mit dem Benutzer über die Kommandozeile oder über eine grafische Bedienoberfläche erfolgen?
Im Finalen Produkt soll die Interaktion mit dem User über eine grafische Bedienoberfläche innerhalb der App erfolgen. Das Dashboard soll jedoch nur prototypisch umgesetzt werden. Wenn man sich aber den Nutzen einer fertigen grafischen Oberfläche gegenüber der Kommandozeile ansieht, ist eine Interaktion auch innerhalb des Prototyps über eine grafische Oberfläche von Vorteil, da sowieso das GUI programmiert wird und dadurch alles in einem Fenster ist, was das Testen aus der Sicht eines Benutzers deutlich einfacher macht.

4.5 Testprogramme

Die Machbarkeit aller oberhalb angeführtern Punkte wird durch kleine Testprogramme getestet und wird als durchführbar eingestuft. Da die länge der Dokumentation zu lange werden würde, wenn auf jedes einezelne Testprogramm eingegegangen werden würde sind diese Testprogramme unter https://github.com/Lukilein52/DocumentationPhase1ProjectPython/tree/master/Phase1%20-%20Konzeptionsphase/Testprogramme zu finden.